

Create Table *Products*

```
SQL> create table products (product_id number(3) primary key,  
1 name varchar(20),  
2 price number(10,2),  
3 stock_quantity number(20),  
4 category varchar(20));
```

Table created

```
SQL> insert into products (product_id, name, price, stock_quantity,  
category) values (1, 'Laptop', 800.00, 15, 'Electronics');
```

row inserted

Inserted 10 rows

Product_id	name	price	Stock_quantity	category
1	Laptop	800.00	15	Electronics
2	Smartphone	600.00	25	Electronics
3	Headphones	150.00	40	Accessories
4	Smartwatch	200.00	30	Wearables
5	Tablet	350.00	20	Electronics
6	Gaming Mouse	50.00	50	Accessories
7	Keyboard	70.00	35	Accessories
8	Monitor	250.00	18	Electronics
9	Printer	120.00	10	Office
10	External HDD	80.00	45	Storage

1. BETWEEN ... AND (Range Condition)

Used to filter results where a column's value falls within a specified range.

```
SQL> select name from products where price between 250 and 600;
```

output

Name
Smartphone
Tablet
Monitor

Retrieves products with prices between 250 and 600.

Sorting results in increasing order:

```
SQL> select name from products where price between 250 and 600 order  
by product_id;
```

output

Name
Smartphone
Tablet
Monitor

Orders results in increasing order based on product_id.

Sorting results in increasing order:

```
SQL> select name from products where price between 250 and 600 order  
by product_id desc;
```

output

Name
Monitor
Tablet
Smartphone

Orders results in descending order based on product_id.

2. IN (Match Multiple Values)

Used to filter results where a column matches any value in a given list.

```
SQL> select name from products where stock_quantity in (40, 30, 25);
```

output

Name
Smartphone
Headphones
Smartwatch

Retrieves products that have stock quantities of 40, 30, or 25.

3. LIKE (Pattern Matching)

Used to search for a specific pattern in text data.

```
SQL> select name, price from products where category like 'E%';
```

output

Name	Price
Laptop	800.00
Smartphone	600.00
Tablet	350.00
Monitor	250.00

Retrieves products where the category starts with "E".

```
SQL> select name, price from products where category like '%E';
```

output

Name	Price
Printer	120.00
External HDD	80.00

Retrieves products where the category ends with "E".

```
SQL> select name, price from products where category like '%s%';
```

output

Name	Price
Laptop	800.00
Smartphone	600.00
Headphones	150.00
Smartwatch	200.00
Tablet	350.00
Gaming Mouse	50.00
Keyboard	70.00
Monitor	250.00

Retrieves products where the category contains "s" anywhere.

4. SUM() (Aggregate Function)

Calculates the total sum of a numeric column.

```
SQL> select sum(price) from products;
```

output

Sum(price)
2670

Retrieves the total sum of all product prices.

5. GROUP BY (Grouping Data)

Used to group results by a specific column when using aggregate functions.

```
SQL> select category, sum(price) from products; ❌
```

```
SQL> select sum(price) from products group by category;
```

output

Sum(price)
270
120
2000
200
80

Groups products by category and calculates the total price for each.

6. HAVING (Filter Groups)

Used to filter grouped data after aggregation.

```
SQL> select category, sum(price) from products group by category  
having category = 'Electronics';
```

output

Category	Sum(price)
Electronics	2000

Retrieves only the total price for the "Electronics" category.